

网络优化与正则化

数据科学与大数据技术专业

第 7 讲

1 导读

除本导读外, 本讲共有 8 个主体节. 它们围绕同一条主线展开: 先判断深度网络训练中到底是 optimization 问题还是 generalization 问题, 再分别讨论优化算法、初始化、结构归一化、正则化、损失与数据、超参数搜索, 最后说明如何把这些实验流程交给 Agent 自动化执行.

1. **网络优化的特点:** 说明深度网络训练为什么不同于普通低维凸优化, 重点区分训练损失、泛化误差、高维非凸地形、鞍点和平坦解.
2. **优化算法改进:** 从学习率、batch size、Momentum、RMSProp、Adam 和梯度截断等角度说明优化器如何改变参数更新方向与步长尺度.
3. **参数初始化与预训练:** 说明初始化如何打破神经元对称性并控制层间信号尺度, 以及预训练为什么可以作为更有语义的初始点.
4. **网络结构与归一化:** 说明结构设计如何把任务先验写进函数集合, 归一化和残差连接如何让深层网络更容易训练.
5. **正则化与泛化控制:** 说明 L_1/L_2 正则化、weight decay、Dropout、早停和 SGD 噪声如何约束模型, 使低训练损失解更可能泛化到新数据.
6. **损失函数与数据层面的改进:** 说明为什么训练目标不一定等于评价指标, 以及交叉熵、标签平滑、数据增强、Mixup 和半监督项如何改变模型接收到的学习信号.
7. **超参数优化:** 说明如何用训练集、验证集和测试集建立规范的模型选择流程, 并用网格搜索、随机搜索等方法系统比较配置.
8. **AutoResearch:** 说明 Agent 如何在明确边界内自动提出实验、修改代码、运行训练、读取验证结果并保留或回滚改动.

如果细节都忘了: 最应该记住的是, 深度学习训练要先看 training loss 和 validation loss 的关系. Training loss 降不下去, 优先检查优化器、学习率、batch size、初始化、归一化和网络结构; training loss 很低但 validation loss 不好, 才优先考虑正则化、数据增强、早停、模型容量和更可靠的验证协议. 本讲后面的所有方法, 本质上都是在这条诊断线上帮助我们更稳地训练模型, 或更可靠地选择能泛化的模型.

问题思考:

- 当一次训练实验失败时, 你会先根据 training loss 和 validation loss 判断哪一类问题? 为什么?

2 网络优化的特点

本节导读:

- **本节目的:** 看清深度网络训练和普通低维优化有什么不同, 避免把所有训练问题都混成一句“模型没调好”.
- **为什么学:** 后面的优化器、初始化、归一化和正则化都在回应同一个问题: 如何在训练集上找到低损失解, 同时让这个解在新数据上仍然稳定.
- **核心内容:** 深度学习训练不是单纯求一个全局最优参数, 而是在高维非凸空间中寻找训练损失低、梯度传播稳定、泛化误差可控的区域.
- **最小例子:** 考虑一个 d 维参数空间中的驻点, 若每个方向的曲率近似独立, 且“向上弯”和“向下弯”的机会都不小, 那么成为局部极小值需要所有 d 个方向都向上弯, 概率会随 d 指数级变小. 只要有某些方向向上弯、某些方向向下弯, 这个点就是鞍点; 因此在高维非凸优化中, 训练更常遇到的是鞍点或平坦停滞区域, 而不是坏的孤立局部极小值.
- **最该记住:** 先看 training loss 和 validation loss 的相对关系, 再判断问题主要来自 optimization 还是 generalization.

前面几讲已经说明, 神经网络训练的基本机制仍然是用损失函数定义目标, 再利用梯度信息更新参数. 但深度学习中的优化问题并不是普通低维凸优化的简单放大版: 参数维度高、目标函数非凸、网络结构差异大, 并且训练误差与泛化能力之间还存在持续的取舍.

1. 从经验风险到泛化误差

- 机器学习真正希望最小化的是期望风险

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim P_{\text{data}}} [\mathcal{L}(y, f(\mathbf{x}))],$$

也就是模型在真实数据分布上的平均损失. 由于真实分布 P_{data} 不可直接获得, 实际训练中只能最小化训练集上的经验风险

$$\hat{\mathcal{R}}_D(f) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)})).$$

- 经验风险对应 optimization: 训练算法要让 training loss 尽可能降下来. 期望风险对应 generalization: 模型还要在未见数据上保持稳定表现. 二者之间的差距就是泛化误差的来源.
- 当模型容量很大而训练数据有限时, 单纯追求经验风险最小可能使模型记住训练集中的噪声, 造成 overfitting. 正则化的作用就是把先验偏好写入训练目标, 例如

$$\mathcal{J}(\theta) = \hat{\mathcal{R}}_D(\theta) + \lambda \Omega(\theta).$$

这通常会提高训练集损失, 但可能降低验证集和测试集损失.

- 需要注意, “简单模型泛化更好”不是绝对规律. 模型过于简单会欠拟合, 模型过于复杂又可能过拟合. 深度学习训练的核心问题之一, 正是在足够表达能力和可控泛化误差之间取得平衡.

2. 高维非凸优化

- 在低维凸优化中, 若目标函数满足合适条件, 梯度下降可以收敛到全局最优点. 但深度神经网络的目标函数通常是关于大量参数 θ 的非凸函数, 因而只能期望优化算法找到足够好的低损失区域.
- CNN、RNN、Transformer 等结构差异很大, 其参数耦合方式、梯度传播路径和有效 batch size 都不同. 因此, 很难存在一组对所有网络都最优的训练配置. 网络层数、隐藏单元数、卷积核大小、残差块设计、batch size、学习率和学习率调度都可能显著改变训练结果.
- 深度模型的参数量通常很大, 数据规模也很大. 完整二阶方法需要存储或近似 Hessian 矩阵, 代价往往过高; 实践中更常使用 Mini-Batch SGD、Momentum、Adam 等一阶方法. 这类方法单步成本较低, 但对学习率、初始化和梯度尺度非常敏感.
- 不同初始化可能把优化过程带到不同区域, 最终得到训练损失接近但泛化表现不同的解. 因此, 参数初始化不是训练前的细节, 而是影响优化路径和最终解性质的重要因素.

3. 鞍点比坏的局部极小值更常见

- 对非凸函数, 梯度为零的驻点可能是局部极小值、局部极大值, 也可能是鞍点. 一维图像中最容易看到的是多个局部极小值, 但高维空间中的情况并不完全相同.
- 一个启发式理解是: 若某点在每个方向上都呈现“向上弯”的概率近似为 p , 那么在 d 个相互独立方向上同时成为严格局部极小值的概率约为 p^d . 当 $d = 10000$ 时, 这个概率会迅速变得很小. 这个例子不是严格定理, 但说明在高维非凸曲面中, 大量驻点更可能具有某些下降方向, 即表现为鞍点.
- 鞍点附近梯度可能很小, 使训练曲线短时间停滞; 但只要存在负曲率方向, 合适的随机扰动、mini-batch 噪声、Momentum 或学习率调度都有机会帮助参数离开该区域. 因此, training loss 降不下去时, 不能马上归因于 overfitting, 还要检查是否存在优化停滞、学习率不合适或梯度尺度异常.

4. 参数冗余与平坦极小值

- 深度网络往往是过参数化的: 许多不同参数可以表示相同或近似相同的函数. 例如对没有额外破坏尺度关系的 ReLU 网络, 若第 l 层权重乘以常数 $c > 0$, 而下一层相应权重除以 c , 利用 ReLU 的正齐次性, 网络整体函数可以保持不变.
- 这种冗余意味着“最优解”通常不是孤立的一个点, 而可能是一片训练损失接近的区域. 若某个解的邻域内损失变化较慢, 就称它位于平坦极小值 (Flat Minima) 附近. 平坦区域中的参数对小扰动更不敏感, 因而常被认为与更稳定的泛化表现有关.
- 但“平坦”本身依赖参数化方式, 不能简单理解为越平越好, 也不能说所有局部极小值都等价. 更合理的说法是: 在深度网络中, 优化的目标通常不是寻找唯一全局最优参数, 而是找到训练损失低、对抗动较稳定、泛化表现可接受的解.

5. 梯度传播与优化地形

- 优化地形指高维参数空间中损失函数曲面的形状. 它不仅包含局部极小值和鞍点, 也包含不同方向曲率差异、谷底是否狭长、低损失区域是否连通等信息.
- 深层网络还有一类特殊困难: 梯度需要沿许多层反向传播. 若层间 Jacobian 的有效尺度长期小于 1, 梯度会逐层衰减; 若长期大于 1, 梯度可能爆炸. 因此, 梯度消失和梯度爆炸都与网络结构、激活函数、初始化和归一化方式有关.

- 残差连接 (skip connection) 是改善优化地形的重要结构. 一个残差块可写为

$$\mathbf{a}^{(l+1)} = F(\mathbf{a}^{(l)}; \mathbf{W}^{(l)}) + \mathbf{a}^{(l)}.$$

旁路项提供了接近恒等映射的信息通道, 也让梯度在反向传播时有更直接的路径.

- 因此, 残差连接不只是“防止梯度消失”的技巧. 它改变了网络架构和参数空间的几何性质, 通常需要与合适初始化、归一化、优化器和学习率调度共同使用.

2.1 训练技巧的诊断框架

训练神经网络时, 一个方法是否“有效”不能只看最终分数是否变高. 更重要的是先判断它改变了机器学习流程中的哪一个环节, 再判断它主要解决 optimization 还是 generalization 问题. 这个诊断框架可以避免把所有训练失败都笼统归因于过拟合.

1. 从三步学习流程定位方法

- 现代监督学习仍可概括为三步: 定义损失函数 $\mathcal{L}$, 规定函数集合 $\mathcal{F}$, 再通过优化算法在 $\mathcal{F}$ 中寻找经验风险较低的函数.
- 神经网络通常把 $\mathcal{F}$ 画得很大, 因而具有更强表达能力; 但表达能力强并不等于一定容易训练, 也不等于一定泛化更好.
- 因此, 每遇到一个训练技巧时都应先问: 它是在改 loss, 在改 network architecture, 还是在改搜索参数的 optimizer?

2. 区分 optimization 与 generalization

- Optimization 关注训练集上的 loss 是否能够降下去. 若一个深层网络的 training loss 甚至高于线性模型, 这通常不是过拟合, 而是优化失败: 因为深层网络的函数集合原则上包含线性模型能表示的函数.
- Generalization 关注训练集和验证集之间的差距. 若 training loss 已经足够低, 但 validation loss 明显较高, 这才是典型过拟合问题.
- 两类问题需要不同工具. 当 training loss 降不下去时, 应优先检查学习率、优化器、初始化、归一化、残差连接等; 当 training loss 很低而 validation loss 不佳时, 才应优先考虑正则化、数据增强、Dropout、权重衰减和更合适的模型容量.

3. 常见方法的定位表

- 下表给出本讲后续内容的基本分类. 需要注意, 有些方法同时影响多个目标, 但通常仍有主次之分.

方法	主要改变	主要收益
Adam, RMSProp, Momentum	优化算法	Optimization
Learning rate schedule	优化算法	Optimization
Dropout	训练过程	Generalization
Xavier/He 初始化, 预训练	初始参数	Optimization, Generalization
CNN, 参数共享	函数集合	Generalization
残差连接, 归一化层	网络结构	Optimization 为主
Cross-Entropy, Hinge Loss	损失函数	Optimization 或 Generalization
数据增强, 半监督学习	损失中的数据项	Generalization
L_1/L_2 正则化, AdamW	损失或优化过程	Generalization

4. 验证集的角色

- 模型训练完成后不能直接在测试集上反复试错, 而应先使用验证集判断模型选择、超参数选择和训练策略是否合理.
- 验证集结果若不理想, 应回到三步流程中定位原因: 是 loss 不合适, 函数集合不合适, 还是优化过程没有找到足够好的参数.
- 测试集只应在最终模型确定后使用, 否则频繁根据测试集调参会把测试集也变成训练过程的一部分, 导致性能估计过于乐观.

问题思考:

- 为什么 training loss 很低并不等于模型具有好的泛化能力?
- 为什么 deep network 的 training loss 高于 linear model 时, 通常应先怀疑 optimization 而不是 overfitting?
- 一个新训练技巧若让 training loss 更低, 是否一定能让 validation loss 更低? 这与 optimization 和 generalization 的区别有什么关系?

3 优化算法改进

本节导读:

- **本节目的:** 说明深度学习中的优化器到底在改什么: 有的主要修正更新方向, 有的主要修正不同坐标上的步长, 有的同时做这两件事.
- **为什么学:** 同一个网络在同一批数据上可能因学习率、batch size、动量或自适应尺度不同而表现出完全不同的训练曲线.
- **核心内容:** 一阶优化算法都在利用带噪声的梯度估计决定参数更新, 训练稳定性取决于方向估计、步长尺度和随机噪声之间的平衡.
- **最小例子:** 在一个狭长谷底中, 普通 SGD 会在陡峭方向来回震荡而在平坦方向前进很慢; Momentum 会积累持续方向, RMSProp 会压低长期梯度较大坐标的有效步长, Adam 则把二者结合起来.

- **最该记住:** 学习率决定全局步子大小, optimizer 决定如何利用历史梯度和坐标尺度, batch size 决定梯度噪声强弱.

优化算法对应三步学习流程中的第三步: 在已经确定 loss 和函数集合后, 寻找较好的参数 θ . 深度网络的损失曲面通常高维、非凸、尺度差异大, 因而优化器既要决定沿哪个方向走, 也要决定每一步走多远.

1. 统一的更新视角

- 设第 t 次迭代的参数为 θ_t , 损失函数为 $\mathcal{L}(\theta)$, 梯度为

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}(\theta_t).$$

大部分一阶优化算法都可以理解为先由历史梯度构造一个更新方向, 再用某种尺度因子调整每个坐标的步长:

$$\theta_{t+1} = \theta_t + \Delta \theta_t, \quad \Delta \theta_t = -\frac{\alpha_t}{\sqrt{\mathbf{G}_t + \epsilon}} \odot \mathbf{M}_t,$$

其中除法、平方根和 $\odot$ 都按元素计算. α_t 是全局学习率, $\mathbf{M}_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t)$ 表示用于更新的方向估计, $\mathbf{G}_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t)$ 表示用于调节各坐标步长的尺度估计.

- 这个公式不是要求所有优化器都写成完全相同的实现, 而是提供一个诊断框架: SGD 主要使用当前梯度, Momentum 修改方向估计 $\mathbf{M}_t$, Adagrad 和 RMSProp 修改尺度估计 $\mathbf{G}_t$, Adam 同时修改二者.
- 最简单的梯度下降对应 $\mathbf{M}_t = \mathbf{g}_t$, $\mathbf{G}_t = \mathbf{1}$, 更新为

$$\theta_{t+1} = \theta_t - \eta \mathbf{g}_t.$$

其局限在于所有参数共用同一个学习率. 若某些方向曲率很大, 学习率稍大就会震荡或发散; 若某些方向曲率很小, 学习率稍小又会移动过慢.

2. 小批量随机梯度下降与 batch size

- 全量梯度下降每次使用全部训练样本, 梯度估计稳定但计算代价高. 随机梯度下降每次只用单个样本, 计算快但噪声大.
- 小批量随机梯度下降 (Mini-Batch SGD) 每次抽取 B 个样本, 用

$$\mathbf{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \mathcal{L}^{(i)}(\theta_t)$$

近似全量梯度. 它兼顾并行效率、内存开销和梯度噪声, 因而是深度学习训练的默认形式.

- 若 mini-batch 是从训练集均匀抽样得到的, 则随机梯度的期望等于全量梯度:

$$\mathbb{E}[\mathbf{g}_t] = \nabla_{\theta} \hat{\mathcal{R}}_D(\theta_t).$$

因此, batch size 主要改变的是梯度估计的方差, 而不是其期望.

- 用一个坐标上的随机梯度 $X_1, \dots, X_B$ 作直观说明. 若它们近似独立同分布, 均值为 μ , 方差为 σ^2 , 则 batch 平均梯度 $\bar{X} = \frac{1}{B} \sum_i X_i$ 满足

$$\mathbb{E}[\bar{X}] = \mu, \quad \text{Var}(\bar{X}) = \frac{\sigma^2}{B}.$$

这解释了为什么 batch 越大, 梯度曲线通常越平滑; batch 越小, 噪声越强, 训练曲线越容易振荡.

- 一个 epoch 表示训练集样本大致被使用一遍. 若训练集大小为 N , batch size 为 B , 则每个 epoch 约包含

$$\left\lceil \frac{N}{B} \right\rceil$$

次参数更新. 因此, 按 iteration 比较时, 大 batch 的每一步更稳定; 按 epoch 比较时, 小 batch 在同样样本遍历次数内会更新更多步. 如果不同 batch size 还配了不同学习率, 图上的训练曲线就不能被当作严格控制变量实验来解释.

- batch size 的选择通常需要折中. 较大 batch 能提高 GPU 利用率, 也允许使用较大学习率, 但每个 epoch 的更新次数更少, 梯度噪声更小, 有时会削弱探索性或泛化表现. 较小 batch 噪声更强, 可能带来类似正则化的效果, 但硬件效率和训练稳定性可能下降.

3. batch size 与线性缩放规则

- batch size 和学习率不能分开看. 当 batch size 变大时, 梯度估计的方差减小, 单步更新方向更稳定, 因而常可使用更大的学习率. 反过来, 若 batch 很小而学习率过大, 噪声和步长叠加后容易导致发散.
- 线性缩放规则给出一个常用近似: 若 batch size 从 B 增大到 cB , 学习率可从 η 增大到 $c\eta$. 一个简单推导如下. 假设小 batch 连续做 c 次更新, 且这 c 次梯度近似相同, 则

$$\theta_{t+c} \approx \theta_t - \eta \sum_{j=1}^c g_{t+j} \approx \theta_t - c\eta g.$$

若把这 c 个小 batch 合成一个大 batch, 只做一次更新, 大 batch 梯度近似为同一个 g , 为了得到相近位移, 需要

$$\theta_{t+1} \approx \theta_t - \eta' g \approx \theta_t - c\eta g,$$

因此 $\eta' = c\eta$.

- 这个推导依赖“短时间内梯度变化不大”的近似, 在训练初期或学习率很大时未必成立. 所以大 batch 训练常配合 learning rate warmup: 先用较小学习率启动, 再逐渐升到目标学习率, 以避免随机初始化阶段的大梯度和大学习率共同造成不稳定.

4. 学习率调度与 warmup

- 学习率控制每次更新的全局步长. 学习率太小会让训练需要大量迭代; 学习率合适时可以稳定下降; 学习率太大则可能在谷底两侧震荡, 甚至使 loss 发散.
- 学习率不必在整个训练过程中保持常数, 可写为随时间变化的 η_t . 常见策略包括 step decay, exponential decay, cosine decay 和周期性学习率. 实践中也常根据验证集指标是否停滞来降低学习率.

- 训练早期常使用 warmup: 先用较小学习率开始, 再逐渐增大学习率. 这有助于优化器在矩估计尚不稳定、梯度尺度尚未进入正常范围时避免剧烈更新, 尤其常见于大 batch 和大模型训练.
- 训练中后期常使用 decay: 逐渐减小学习率, 使参数从大步探索转向小步收敛. 周期性学习率或 warm restart 会在局部阶段重新增大学习率, 其直观作用是帮助参数离开尖锐区域或停滞区域; 但它只是经验性策略, 不能保证一定找到更平坦或更优的解.

5. Adagrad, RMSProp 与 Adadelta: 给不同参数不同学习率

- 标准梯度下降对所有坐标使用同一个学习率, 但不同参数维度的梯度尺度和曲率可能差异很大. 自适应学习率方法的基本思想是: 梯度长期较大的坐标使用较小有效步长, 梯度长期较小的坐标使用较大有效步长.
- Adagrad 根据历史梯度平方和调整每个坐标的有效学习率:

$$\mathbf{G}_t = \sum_{\tau=1}^t \mathbf{g}_\tau \odot \mathbf{g}_\tau, \quad \Delta \boldsymbol{\theta}_t = -\eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t + \epsilon}}.$$

- Adagrad 的限制是 $\mathbf{G}_t$ 单调增大, 有效学习率会不断变小. 若早期没有找到足够好的区域, 后期步长可能过小, 使算法难以继续移动.
- RMSProp 使用指数滑动平均替代累积和:

$$\mathbf{G}_t = \beta \mathbf{G}_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t = (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau \odot \mathbf{g}_\tau, \quad \Delta \boldsymbol{\theta}_t = -\eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t + \epsilon}}.$$

这使得很久以前的梯度影响逐渐衰减, 因而有效学习率既可以变小, 也可以在局部梯度变小时重新变大.

- Adadelta 进一步用参数更新量的滑动平均替代固定学习率的一部分. 其直观目标是让更新步长本身也更平滑, 减少手动选择全局学习率的压力. 在现代实践中, RMSProp 和 Adam 更常作为默认候选.

6. Momentum 与 Nesterov: 修正更新方向

- 在小 batch 训练中, 每一步梯度都只是全量梯度的带噪声估计. 如果完全依赖当前梯度, 参数会在某些方向上来回抖动. Momentum 用最近一段时间的平均趋势替代单步梯度:

$$\Delta \boldsymbol{\theta}_t = \rho \Delta \boldsymbol{\theta}_{t-1} - \eta \mathbf{g}_t = -\eta \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau.$$

其中 ρ 控制历史更新保留程度. 若多个连续梯度方向一致, 动量会加速前进; 若某个方向梯度来回变号, 移动平均会相互抵消, 从而减少震荡.

- 从物理类比看, 参数变化率可看作速度, $-\eta \mathbf{g}_t$ 可看作当前梯度带来的加速度, ρ 则控制惯性和阻尼. 这个类比有助于理解直觉, 但优化算法本质上仍是在数值上构造更稳定的下降方向.
- Nesterov Accelerated Gradient (NAG) 先沿当前动量方向向前看一小步, 再在前瞻位置计算梯度:

$$\hat{\boldsymbol{\theta}}_t = \boldsymbol{\theta}_t + \rho \Delta \boldsymbol{\theta}_{t-1}, \quad \Delta \boldsymbol{\theta}_t = \rho \Delta \boldsymbol{\theta}_{t-1} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\hat{\boldsymbol{\theta}}_t).$$

与普通 Momentum 相比, Nesterov 使用“预计会到达的位置”来估计梯度, 因而在某些情况下能更早修正过大的惯性方向.

7. Adam: 方向修正与尺度修正的结合

- Adam 可粗略理解为 Momentum 与 RMSProp 的结合. 它同时维护一阶矩估计和二阶矩估计:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t.$$

- 由于 $\mathbf{m}_t$ 和 $\mathbf{v}_t$ 初始为零, 完整 Adam 会使用 bias correction:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t},$$

并更新

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}.$$

- Adam 往往是深度学习实验的默认起点, 因为它对梯度尺度和学习率选择较稳健. 但 Adam 主要改善 optimization, 不会自动解决 overfitting. 当 training loss 已经很低而 validation loss 很高时, 更直接的工具通常是正则化、数据增强、早停或模型容量调整.

8. 梯度截断

- 前面的方法主要讨论如何更快、更稳地下降; 梯度截断则针对梯度爆炸. 当 $\|\mathbf{g}\|$ 很大时, 一次更新可能把参数带离合理区域, 尤其常见于循环神经网络和大模型训练.
- 常用的 norm clipping 写为

$$\tilde{\mathbf{g}} = \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|_2}\right).$$

若梯度范数不超过阈值 c , 则保持原梯度; 若超过 c , 则把梯度缩放到范数为 c . 这样可以限制单次参数更新的最大幅度.

- 梯度截断是一种有效的工程性稳定手段, 但它不改变损失函数, 也不从根本上消除导致梯度爆炸的结构原因. 因此, 它常与合适初始化、归一化、残差连接和学习率调度配合使用.

问题思考:

- 为什么 batch size 增大会降低随机梯度的方差, 但不一定使训练和泛化都更好?
- 线性缩放规则依赖哪些近似条件? 为什么大 batch 训练常需要 warmup?
- 在统一更新公式中, Momentum、RMSProp 和 Adam 分别主要改变了哪一部分?

4 参数初始化与预训练

本节导读:

- 本节目的:** 解释训练开始前的参数选择为什么会影响整个优化路径, 而不是一个可以随意忽略的工程细节.
- 为什么学:** 深层网络的信号和梯度要穿过很多层, 初始尺度不合适会在训练真正开始前就造成对称性、梯度消失或梯度爆炸问题.

- **核心内容:** 好的初始化既要打破同层神经元之间的对称性, 又要让前向激活和反向梯度在层间保持合理尺度; 预训练则是在随机尺度假设之外, 用额外数据给优化过程一个更有语义的起点.
- **最小例子:** 若两个隐藏单元的入边、偏置和出边权重完全相同, 它们的输出和梯度会一直相同, 这两个单元名义上不同, 实际上只能学习同一个特征.
- **最该记住:** 初始化首先要破坏对称性, 其次要控制尺度; Xavier、He、正交初始化和预训练只是围绕这两个目标的不同实现.

初始化决定优化过程从损失曲面的哪个位置出发. 在深度网络中, 即使用同一个模型、同一批数据和同一个优化器, 不同随机初始化也可能导致明显不同的训练曲线和最终解. 因此, 初始化既不是纯工程细节, 也不是越随机越好; 它需要同时打破神经元之间的对称性, 并控制信号和梯度在层间传播时的尺度.

1. 为什么不能把同层神经元初始化成完全相同

- 参数初始化首先要打破对称性. 若同一层的多个神经元具有完全相同的入边权重、偏置和出边权重, 它们在前向传播中会产生相同输出, 在反向传播中也会得到相同梯度, 因而后续更新仍然保持相同.
- 用一个含两个隐藏单元的最小例子可以看清这一点. 设输入为标量 x , 隐藏层为

$$h_1 = \phi(wx + b), \quad h_2 = \phi(wx + b),$$

输出层为

$$\hat{y} = v_1 h_1 + v_2 h_2, \quad v_1 = v_2 = v.$$

对任意损失 $\mathcal{L}(y, \hat{y})$, 记 $\delta = \partial \mathcal{L} / \partial \hat{y}$. 因为 $h_1 = h_2 = h$, 两个输出权重的梯度相同:

$$\frac{\partial \mathcal{L}}{\partial v_1} = \delta h_1 = \delta h = \delta h_2 = \frac{\partial \mathcal{L}}{\partial v_2}.$$

两个输入权重的梯度也相同:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \delta v \phi'(wx + b)x = \frac{\partial \mathcal{L}}{\partial w_2}, \quad \frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial b_2}.$$

因此一次梯度下降后仍有 $w_1 = w_2, b_1 = b_2, v_1 = v_2$. 这两个隐藏单元虽然名义上是两个神经元, 实际上始终学习同一个特征, 表达能力退化得像一个神经元.

- 全零初始化是这种对称性问题的极端情形. 在多层网络中, 权重矩阵不能整层初始化为零; 偏置项可以初始化为零, 因为随机权重已经能打破同层神经元的对称性.

2. 随机初始化与尺度选择

- 最朴素的随机初始化可以从高斯分布或均匀分布采样, 例如

$$W_{ij} \sim \mathcal{N}(0, \sigma_w^2), \quad W_{ij} \sim \text{Unif}[-r, r].$$

真正关键的问题不是“高斯还是均匀”, 而是方差 σ_w^2 或区间半径 r 应该取多大.

- 若权重尺度太小, 前向激活和反向梯度会逐层衰减; 若权重尺度太大, 激活值可能进入 Sigmoid 或 Tanh 的饱和区, 导致导数接近 0, 也可能使 ReLU 网络中的激活和梯度逐层放大. 初始化的基本目标就是让每一层的输出方差和梯度方差都保持在合理范围内.

- 对于输入维度为 n_{in} , 输出维度为 n_{out} 的线性层, Xavier 初始化常令

$$\text{Var}(W_{ij}) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}},$$

适合 Tanh、Sigmoid 等近似对称的激活函数. 对应的均匀初始化可取

$$W_{ij} \sim \text{Unif} \left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right].$$

- 对于 ReLU 及其变体, 由于大约一半负半轴输出被置零, He 初始化常令

$$\text{Var}(W_{ij}) \approx \frac{2}{n_{\text{in}}}.$$

这可以更好地保持前向信号和反向梯度的尺度.

- 需要注意, 初始化公式依赖激活函数、层类型和实现约定. 在卷积层中, n_{in} 通常等于输入通道数乘以卷积核面积. 因此, 固定使用 $\mathcal{N}(0, 0.01^2)$ 这样的经验常数并不总是合理.

3. 范数保持与正交初始化

- 方差缩放初始化关注的是方差在平均意义下不要逐层爆炸或消失. 正交初始化则进一步从范数保持 (Norm-Preserving) 的角度理解梯度传播.
- 考虑一个 L 层等宽线性网络

$$\mathbf{y} = \mathbf{W}^{(L)} \mathbf{W}^{(L-1)} \dots \mathbf{W}^{(1)} \mathbf{x}.$$

记误差项

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}},$$

即损失对第 l 层净激活值 $\mathbf{z}^{(l)}$ 的梯度. 在线性层中反向传播满足

$$\delta^{(l-1)} = \left(\mathbf{W}^{(l)} \right)^\top \delta^{(l)}.$$

若 $\mathbf{W}^{(l)}$ 为正交矩阵, 即

$$\left(\mathbf{W}^{(l)} \right)^\top \mathbf{W}^{(l)} = \mathbf{I},$$

则有

$$\left\| \delta^{(l-1)} \right\|_2 = \left\| \left(\mathbf{W}^{(l)} \right)^\top \delta^{(l)} \right\|_2 = \left\| \delta^{(l)} \right\|_2.$$

这说明, 在纯线性且等宽的理想情形下, 正交权重不会单纯因为反复矩阵乘法而放大或缩小误差项.

- 实现正交初始化时, 常先采样一个高斯随机矩阵

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top,$$

再取 $\mathbf{U}$ 、 $\mathbf{V}$ 或 $\mathbf{U} \mathbf{V}^\top$ 的适当部分作为权重矩阵. 这里的本质是对随机矩阵做正交化, 不是把初始化问题简单化为求某个对称矩阵的特征向量.

- 正交初始化常用于很深的网络或循环神经网络的隐藏到隐藏连接, 因为 RNN 会在时间维

度上反复乘以同一类矩阵。但在非线性激活、门控、归一化和残差连接同时存在时，正交初始化只能缓解梯度尺度问题，不能单独保证训练稳定。

4. 偏置初始化与结构相关的特殊选择

- 权重初始化负责打破对称性和控制尺度，偏置初始化则更多依赖具体结构。对普通全连接层和卷积层，偏置常初始化为 0，因为随机权重已经使不同神经元接收到不同信号。
- 对 ReLU 网络，早期实践中有时会把偏置设为一个很小的正数，例如 0.01，以降低神经元一开始就完全落在负半轴的概率。但这不是普适规则；使用 He 初始化、Batch Normalization 或合适学习率时，零偏置也很常见。
- 对 LSTM 等循环结构，遗忘门偏置常初始化为正值，使初始阶段更倾向于保留历史状态。这样做有助于长时间依赖中的梯度回传，与正交初始化循环权重的目标是一致的：减少无意义的快速遗忘或梯度衰减。
- 这些策略可以组合使用。例如，一个模型可以对 ReLU 前馈层使用 He 初始化，对循环隐藏矩阵使用正交初始化，对 LSTM 遗忘门使用正偏置，对新加入的分类头使用随机初始化。

5. 预训练作为更强的初始化

- 随机初始化只利用数学尺度假设，预训练 (pre-training) 则利用额外数据和额外任务先学到一组有语义的参数，再把它作为下游任务的初始点。这可以有监督预训练，也可以是自监督或无监督预训练。
- 典型流程是先在大规模数据上训练一个骨干网络，再在目标任务上微调 (fine-tuning)。微调时可以只训练最后几层，也可以让整个网络继续更新，取决于数据量、任务相似度和过拟合风险。
- 例如在图像任务中，可先训练模型预测图片旋转角度，再将得到的表示用于分类任务。这个预训练任务称为 pretext task，真正关心的分类任务称为 downstream task。
- 好的 pretext task 通常需要满足两个条件：第一，数据和标签容易大量构造；第二，任务迫使模型学习对下游任务有用的结构或表示。
- 预训练的特殊之处在于它往往同时帮助 optimization 和 generalization。它让优化从更好的区域开始，也通过大规模数据让表示更稳定、更可迁移。

6. 平坦极小值与初始化的关系

- 即使训练集上存在多个 training loss 很低的解，它们的泛化能力也可能不同。位于尖锐谷底的解对数据分布或参数扰动很敏感，验证集损失可能显著升高。
- 位于较平坦区域的解通常对小扰动更稳定，更可能让 training loss 与 validation loss 接近。这也是为什么初始化、优化器和学习率调度会间接影响泛化表现。
- 但“平坦”本身依赖参数化方式，不能机械理解为越平越好。更重要的是理解：optimization 找到低训练损失只是第一步，解的稳定性同样重要。

问题思考：

- 在两个隐藏单元完全对称的例子中，为什么一次梯度更新后它们仍然保持相同？
- Xavier 初始化、He 初始化和正交初始化分别试图保持什么量不失控？
- 预训练、ReLU 小正偏置和 LSTM 遗忘门正偏置分别解决的是同一种初始化问题吗？

5 网络结构与归一化

本节导读:

- **本节目的:** 说明网络结构和归一化分别从两个层面改变学习问题: 前者改变可选的函数集合, 后者改变信号和梯度的尺度条件.
- **为什么学:** 深度学习的成功往往不只是“模型更大”, 而是把任务先验写进结构, 再用归一化、残差连接等机制让这个结构可以被稳定训练.
- **核心内容:** 结构设计决定模型偏向哪些函数, 归一化和残差连接则让这些函数更容易通过梯度下降找到.
- **最小例子:** 若用两个输入特征训练线性模型 $y = w_1x_1 + w_2x_2$, 其中 x_1 的量级约为 1, x_2 的量级约为 1000, 同一个学习率会让两个参数看到的梯度尺度很不均衡. 损失等高线会变得狭长, 梯度下降容易来回震荡或前进很慢; 把输入标准化到相近尺度后, 参数训练通常会稳定得多.
- **最该记住:** CNN 等结构主要是把领域知识写进函数集合, BN/LN 等归一化主要是稳定中间表示的尺度和梯度.

网络结构决定函数集合 $\mathcal{F}$. 合适的结构不是简单地把模型做大, 而是利用任务中的先验知识画出“范围较小但包含好函数”的搜索空间. 归一化方法则主要改变输入或中间表示的尺度, 使优化地形更容易处理. 二者经常一起出现: 结构约束帮助 generalization, 归一化和残差连接则常常让 optimization 更稳定.

1. 卷积网络: 用领域知识缩小函数集合

- 对一张 $H \times W$ 的 RGB 图片, 原始输入可看作 $H \times W \times 3$ 个数. 若直接使用全连接层, 每个神经元都连接到所有像素, 参数量会非常大, 也会忽略图像中强烈的局部结构.
- CNN 的第一项约束是局部感受野 (receptive field): 低层神经元只观察图片中的小区域, 因为边缘、角点、纹理等 basic pattern 通常可以由局部信息识别.
- 卷积核大小 (kernel size) 决定单层局部观察范围; 步幅 (stride) 决定窗口移动间隔; 填充 (padding) 决定边界处是否补零. 对输入尺寸 H , 卷积核大小 K , padding 为 P , stride 为 S 的一维情形, 输出尺寸为

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

二维图像可分别在高和宽两个方向应用同样公式.

- CNN 的第二项约束是参数共享 (parameter sharing): 同一个 filter 在不同空间位置复用. 这表达了一个重要先验: 同一种局部模式可能出现在图片的不同位置.
- 因此, CNN 并不是比全连接网络“更复杂”的函数集合, 而是在全连接层中加入局部连接和参数共享后得到的更小集合. 它的好处主要是提升图像等结构化数据上的 generalization, 而不是直接保证 optimization 更容易.

2. 残差连接: 让深层网络更容易优化

- 深层网络中, 靠近输入端的参数变化要经过很多层才能影响最终 loss. 这会导致梯度消失; 在某些区域又可能出现梯度爆炸. 真正困难的是二者交替出现, 使学习率既不能太大也不能太小.

- 残差连接 (skip connection) 把一层或一组层写为

$$\mathbf{a}^{(l+1)} = F(\mathbf{a}^{(l)}; \mathbf{W}^{(l)}) + \mathbf{a}^{(l)}.$$

其中 F 表示普通的非线性变换, 旁路项 $\mathbf{a}^{(l)}$ 提供了直接的信息通道.

- 直观上, 残差连接让底层表示和梯度可以沿“捷径”传播, 从而使低层参数对输出保持更稳定的影响. 它主要改善 optimization, 同时也能让深层模型更容易学习接近恒等映射的变换.
- 需要注意, 残差连接改变的是网络架构, 不是 loss 本身. 它让优化地形更友好, 但仍需要配合合适初始化、归一化和学习率.

3. 输入尺度与尺度不变性

- 若算法在缩放全部或部分特征后学习和预测结果基本不变, 可以说它具有较好的尺度不变性. 许多常用算法并不天然满足这一点. 例如最近邻分类器依赖欧氏距离, 若用“长度”和“宽度”预测类别, 其中一个维度被放大 1000 倍, 距离几乎会被该维度主导.
- 线性模型也会受到尺度影响. 对

$$y = w_1 x_1 + w_2 x_2,$$

若 x_1 与 x_2 的量级相差 1000 倍, 同一组初始化尺度和同一个学习率对 w_1, w_2 的影响并不对称. 从函数表示看, 参数可以通过反向缩放来补偿特征尺度; 但从梯度下降看, 损失等高线会变得狭长, 梯度方向常常不是最有效的搜索方向, 收敛需要更多迭代.

- 因此, 输入归一化的目标是让不同特征维度处在可比尺度上. 这不会改变任务本身, 但会改善优化问题的条件数, 使学习率更容易设置, 也让前一节讨论的参数初始化更容易使用统一尺度.

4. 常见输入归一化方法

- 最小最大归一化把每个特征缩放到固定区间. 对第 j 个特征可写为

$$\tilde{x}_j^{(n)} = \frac{x_j^{(n)} - \min_m x_j^{(m)}}{\max_m x_j^{(m)} - \min_m x_j^{(m)} + \epsilon}.$$

其中极值应由训练集估计, 再用于验证集和测试集. 若分母接近 0, 说明该特征在训练集中几乎为常数, 通常可以删除或统一映射为 0.

- 标准化把每个特征做平移和缩放:

$$\mu_j = \frac{1}{N} \sum_{n=1}^N x_j^{(n)}, \quad \sigma_j^2 = \frac{1}{N} \sum_{n=1}^N (x_j^{(n)} - \mu_j)^2,$$

$$\tilde{x}_j^{(n)} = \frac{x_j^{(n)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}.$$

标准化后的特征均值约为 0, 方差约为 1, 但这并不意味着任意分布都会变成正态分布. 它只改变一阶和二阶尺度, 不会自动消除特征之间的相关性.

- 白化 (Whitening) 进一步降低特征之间的相关性. 若中心化后的协方差矩阵分解为 $\Sigma = U\Lambda U^\top$, PCA 白化可粗略写为

$$\tilde{\mathbf{x}} = \Lambda^{-\frac{1}{2}} U^\top (\mathbf{x} - \boldsymbol{\mu}).$$

白化后各方向方差更接近, 且相关性降低; 但它需要估计和分解协方差矩阵, 计算和数值代价都高. 在很多深度学习任务中, 对输入做标准化已经足够.

- 验证集和测试集必须使用训练集上的 μ_j, σ_j 或最大最小值. 若重新使用验证集或测试集自身统计量, 就把评估数据的信息泄漏进了预处理过程.

5. 从输入归一化到中间层归一化

- 输入归一化改善第一层的输入尺度. 同样的想法也可以用于网络中间层. 对第 l 层,

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}).$$

当前面各层参数不断更新时, 第 l 层看到的输入分布也会变化. 早期文献常把这种现象称为内部协变量偏移 (internal covariate shift).

- 更现代的理解是: 归一化层会重参数化网络, 使中间表示的尺度更稳定, 梯度也更稳定, 从而让优化地形更平滑, 允许使用更大的学习率. 因此, 归一化的收益不只来自“分布不变”, 也来自更易优化的参数化方式.
- 常见中间层归一化包括 Batch Normalization, Layer Normalization, Weight Normalization 和早期 CNN 中使用过的 Local Response Normalization. 它们的关键差别在于: 统计量沿哪些维度计算, 训练和推理阶段是否一致, 以及是否依赖同一 batch 中的其他样本.

6. 批量归一化

- Batch Normalization (BN) 在一个 mini-batch 内对每个特征维度估计均值和方差. 给定包含 K 个样本的小批量, 记第 k 个样本在第 l 层的净输入为 $\mathbf{z}^{(k,l)}$, 则

$$\boldsymbol{\mu}_B = \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)},$$

$$\boldsymbol{\sigma}_B^2 = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \boldsymbol{\mu}_B) \odot (\mathbf{z}^{(k,l)} - \boldsymbol{\mu}_B).$$

这里 $\boldsymbol{\mu}_B$ 和 $\boldsymbol{\sigma}_B^2$ 都是向量, 维度等于该层特征数或神经元数.

- 归一化和仿射恢复可写为

$$\hat{\mathbf{z}}^{(k,l)} = \frac{\mathbf{z}^{(k,l)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \epsilon}}, \quad \mathbf{y}^{(k,l)} = \boldsymbol{\gamma} \odot \hat{\mathbf{z}}^{(k,l)} + \boldsymbol{\beta}.$$

上式中的除法、平方根和 $\odot$ 都按元素计算. ϵ 用于避免方差为 0 或非常小时产生数值不稳定. 可学习参数 $\boldsymbol{\gamma}$ 和 $\boldsymbol{\beta}$ 不是为了“保留非线性”本身, 而是为了恢复必要的尺度和偏移, 让网络在需要时可以选择合适的激活区间, 甚至近似还原未归一化的表示.

- BN 的主要收益是提高优化效率. 它让中间层输入尺度更稳定, 使学习率更容易设置, 训练更快更稳. 同时, mini-batch 统计量带有随机性, 一个样本的归一化结果会受同 batch 其他样本影响, 因而 BN 也可能带来一定隐式正则化效果.

- BN 的局限同样来自 batch 统计量. 当 batch size 太小时, 均值和方差估计噪声较大; 在循环神经网络、变长序列或自回归推理中, 跨样本统计量也不总是方便或稳定. 此外, BN 在训练和推理阶段行为不同: 训练时使用当前 mini-batch 统计量, 推理时通常使用训练过程中累计的 running mean 和 running variance.

7. 层归一化与其他归一化

- Layer Normalization (LN) 不依赖 batch 中其他样本, 而是在单个样本内部对同一层的所有特征求均值和方差. 对 $\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$,

$$\mu^{(l)} = \frac{1}{M_l} \sum_{i=1}^{M_l} z_i^{(l)}, \quad (\sigma^{(l)})^2 = \frac{1}{M_l} \sum_{i=1}^{M_l} (z_i^{(l)} - \mu^{(l)})^2,$$

$$\text{LN}_{\gamma, \beta}(\mathbf{z}^{(l)}) = \gamma \odot \frac{\mathbf{z}^{(l)} - \mu^{(l)}}{\sqrt{(\sigma^{(l)})^2 + \epsilon}} + \beta.$$

- 若把一个 batch 的激活排列为矩阵 $\mathbf{Z} \in \mathbb{R}^{K \times M_l}$, 并约定每一行是一个样本、每一列是一个特征, 那么 BN 是沿列统计, 即对同一特征在不同样本上的值归一化; LN 是沿行统计, 即对同一样本内部不同特征归一化. 若采用相反的矩阵摆放约定, “行/列”的说法会反过来, 但“跨 batch”与“样本内”这个本质区别不变.
- LN 更适合 batch size 较小、序列长度变化或自回归推理场景, 在 Transformer 中尤其常用. Weight Normalization 直接重参数化权重向量的长度和方向, 试图把尺度学习与方向学习分开. Local Response Normalization 曾用于早期 CNN, 但在现代网络中较少作为默认选择.
- 选择归一化方法时, 不能只看名称. 应同时考虑网络结构、batch size、训练/推理是否一致、是否需要跨样本统计量以及硬件实现效率.

问题思考:

- 为什么 CNN 可以看作全连接层的受限版本? 这种限制为什么反而有助于图像任务?
- 输入特征尺度差异为什么会同时影响最近邻方法和梯度下降训练?
- 在样本为行、特征为列的矩阵约定下, Batch Normalization 和 Layer Normalization 分别沿哪个方向计算统计量? 它们在训练和推理阶段有什么差别?

6 正则化与泛化控制

本节导读:

- **本节目的:** 解释怎样在模型已经能拟合训练集之后, 进一步控制它在未知数据上的行为.
- **为什么学:** 深度网络常常有足够能力把训练集记住, 但真正有价值的是在验证集和测试集上保持稳定.
- **核心内容:** 正则化不是让 training loss 更低的技巧, 而是通过参数惩罚、随机扰动、数据增强、早停等方式改变学习偏好, 使模型更可能落在泛化较好的解附近.

- **最小例子**: 在线性模型和平方损失下, 梯度下降训练得越久, 参数越容易沿数据中不稳定的方向继续变大; 早停法相当于限制训练时间, L_2 正则化相当于直接惩罚过大的参数. 在这个简单情形中, 二者都能抑制参数过度放大, 因而可以看作两种效果相近的正则化方式.
- **最该记住**: 当 training loss 高时先查优化和模型能力, 当 training loss 低而 validation loss 高时才优先加强正则化.

正则化的目标是在训练集拟合能力和未知数据稳定性之间取得平衡. 它并不是“让模型训练得更好”的同义词; 很多正则化方法会使 training loss 变高, 但可能让 validation loss 下降. 对深度网络尤其要注意: 过参数化只表示参数量远大于样本量或有效约束数量, 并不必然等于泛化能力差. 正则化、数据增强、SGD 噪声、早停和权重衰减等机制, 都是在给学习过程加入某种先验偏好或稳定性约束.

1. 正则化的基本形式与先验偏好

- 设经验风险为 $\hat{\mathcal{R}}(\theta)$, 正则化后的目标可写为

$$\mathcal{J}(\theta) = \hat{\mathcal{R}}(\theta) + \lambda \Omega(\theta),$$

其中 Ω 刻画对参数或函数形状的偏好, λ 控制正则化强度.

- 从约束角度看, 这相当于预先假设可接受的参数或函数位于某个较小区域内. 当两个模型的训练误差接近时, 正则化会偏向参数幅度更小、函数变化更平滑或预测更稳定的解.
- 需要注意, 参数小并不严格等价于函数简单, 过参数化也不自动意味着过拟合. 更准确的说法是: 正则化通过改变可接受解的偏好, 影响优化最后落在哪一类低训练损失区域.

2. ℓ_1 与 ℓ_2 正则化

- 一般的范数正则化可写为

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}; \theta)) + \lambda \ell_p(\theta),$$

其中 p 常取 1 或 2, λ 为正则化系数.

- ℓ_1 正则化取 $\Omega(\theta) = \|\theta\|_1$, 倾向于产生稀疏参数. 从几何直觉看, ℓ_1 约束区域在坐标轴方向有尖角, 最优点更容易落在某些坐标为 0 的位置, 因而常用于特征选择或希望模型更稀疏的场景.
- ℓ_2 正则化取 $\Omega(\theta) = \frac{1}{2} \|\theta\|_2^2$, 倾向于抑制参数过大. 它通常不会把参数精确压到 0, 而是让许多参数同时变小, 从而提高模型对扰动的稳定性.
- 若普通梯度为 $g_t = \nabla \hat{\mathcal{R}}(\theta_t)$, 则带 ℓ_2 正则化的 SGD 更新为

$$\theta_{t+1} = \theta_t - \eta(g_t + \lambda \theta_t) = (1 - \eta\lambda)\theta_t - \eta g_t.$$

上式说明, 在普通 SGD 中 ℓ_2 正则化等价于每次更新时把权重向零收缩一点, 这就是 weight decay 的直观来源.

3. Weight Decay 与 AdamW

- Weight Decay 直接把参数乘上一个小于 1 的衰减系数:

$$\theta_{t+1} = (1 - \beta)\theta_t - \eta g_t.$$

与上一小节的 ℓ_2 更新比较可见, 当 $\beta = \eta\lambda$ 时, 二者在普通 SGD 中形式上等价. 这就是“ ℓ_2 正则化等价于 weight decay”的准确适用条件.

- 但对 Momentum、Adam 这类会改变梯度方向或逐坐标缩放梯度的优化器, 直接把 $\lambda\theta$ 加进梯度会进入动量估计或自适应尺度估计, 不再等同于简单地对参数做乘法衰减.
- AdamW 使用 decoupled weight decay: 先对参数做独立衰减, 再执行 Adam 更新. 形式上可理解为

$$\theta_{t+\frac{1}{2}} = (1 - \eta\lambda)\theta_t, \quad \theta_{t+1} = \text{AdamUpdate}(\theta_{t+\frac{1}{2}}).$$

- 相比 Adam, AdamW 的主要增益是 generalization, 不是更强的 optimization. 因此它常用于 Adam 训练已经顺利但验证集表现不够稳的情形.

4. Dropout

- Dropout 在训练阶段随机丢弃部分神经元或激活值. 对一层表示 $\mathbf{h}$, 若保留概率为 q , 可写为

$$\tilde{\mathbf{h}} = \frac{\mathbf{m} \odot \mathbf{h}}{q}, \quad m_i \sim \text{Bernoulli}(q).$$

这里除以 q 是 inverted dropout 的常见实现, 使 $\mathbb{E}[\tilde{\mathbf{h}}] = \mathbf{h}$, 推理阶段无需额外缩放.

- Dropout 的直观作用是让网络不能过度依赖某些局部神经元组合, 从而降低共适应 (co-adaptation). 如果某个神经元承载了过多信息, 随机丢弃会迫使其他路径也学习可用表示, 使模型对局部扰动更稳定.
- 从集成学习角度看, 每次丢弃都相当于从原网络中采样一个子网络. 若有 d 个可丢弃单元, 理论上存在多达 2^d 个掩码模式. 推理阶段使用完整网络和尺度校正, 可看作对这些子网络预测的一种廉价近似.
- 从贝叶斯近似角度看, Dropout 也可理解为对参数或子网络掩码的随机采样. 若显式做 Monte Carlo Dropout, 可用多次采样近似

$$\mathbb{E}_{q(\theta)} [f(\mathbf{x}; \theta)] \approx \frac{1}{M} \sum_{m=1}^M f(\mathbf{x}; \theta_m).$$

课堂中更重要的是把它理解为训练阶段的随机正则化, 不必把每次普通推理都解释成真实的贝叶斯积分.

- 在循环神经网络中, 若每个时间步都重新采样不同的丢弃掩码, 可能破坏时间维度上的记忆. 因此常使用 locked dropout 或 variational dropout: 对同一序列在所有时间步复用同一个掩码, 使采样到的子网络在时间展开中保持一致.
- 推理阶段通常关闭 Dropout, 使用完整网络. 这体现了训练和推理行为的差异, 与 Batch Normalization 类似都需要在实现中明确区分模式.

5. 早停与 SGD 的隐式正则化

- Early Stopping 使用验证集监控训练过程. 当 validation loss 或验证错误率在若干轮内不再改善时停止训练, 避免模型继续拟合训练集噪声.

- SGD 自身也具有一定隐式正则化效果. 小批量梯度中的噪声会影响参数走向, 有时使模型更容易落在较平坦的区域. 但这种效果依赖 batch size、学习率和数据顺序, 不能替代显式验证.

6. 正则化方法的使用时机

- 如果 training loss 还很高, 过早加入强正则化可能使优化更困难. 此时应先确认模型能否在训练集上学到合理规律.
- 如果 training loss 低而 validation loss 高, 则应优先考虑数据增强、weight decay、Dropout、label smoothing、early stopping 或缩小模型容量.
- 正则化不是越强越好. 过强的正则化会导致欠拟合, 使 training loss 和 validation loss 都较高.

问题思考:

- 为什么 Dropout 常常会提高 training loss, 却仍可能改善 validation loss?
- 在普通 SGD 中, ℓ_2 正则化为什么可以解释为 weight decay? 为什么 Adam 中不能直接照搬这个结论?
- 如果一个模型同时出现 training loss 高和 validation loss 高, 应优先加强正则化还是改善优化? 为什么?

7 损失函数与数据层面的改进

本节导读:

- **本节目的:** 说明训练目标和训练数据本身也可以被设计, 它们会决定模型从每个样本中接收到什么学习信号.
- **为什么学:** 最终评价指标未必可微、原始数据未必足够代表真实分布、标签也未必应该被当作绝对确定的一点质量.
- **核心内容:** 损失函数把任务目标转换成可优化的梯度信号, 数据增强、Mixup、半监督项和标签处理则改变经验风险所看到的数据分布与先验假设.
- **最小例子:** Accuracy 在分类边界不变时对参数小扰动没有反应, 几乎不能给梯度下降提供方向; 交叉熵则会持续鼓励真实类别概率变大, 因而可以作为分类训练的替代目标.
- **最该记住:** 训练 loss 应该可优化并尽量贴近评价目标, 数据层面的改动必须保持任务语义, 否则增强也会变成错误标签.

损失函数对应三步流程中的第一步. 它决定模型在训练时真正被优化的对象. 有时评价指标不能直接作为 loss, 有时则需要通过数据增强、半监督项或标签处理改变 loss 的统计含义.

1. 分类问题为什么不用 accuracy 直接训练

- 分类模型通常输出每个类别的 score, 再取 $\arg \max$ 得到预测类别. Accuracy 只关心预测类别是否正确, 不关心 score 的微小变化.

- 因此, 只要 score 的排序没有跨过决策边界, 参数小幅变化不会改变 accuracy. 从梯度下降角度看, 它在大多数位置的梯度为 0, 难以直接优化.
- 这说明训练 loss 不一定等于最终 evaluation metric. 一个可用的训练 loss 至少应能提供有效梯度, 同时尽量与最终目标保持一致.

2. Softmax 与交叉熵

- 对 C 类分类, 设网络输出 logits 为 $z_1, \dots, z_C$. Softmax 将其转为概率分布:

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)}.$$

- 若真实标签为 one-hot 分布 $\mathbf{y}$, 交叉熵损失为

$$\mathcal{L}_{CE}(\mathbf{y}, \mathbf{p}) = - \sum_{c=1}^C y_c \log p_c.$$

当真实类别为 k 时, 上式化为 $-\log p_k$, 即鼓励模型提高真实类别概率.

- 交叉熵是连续且可微的替代目标, 比 accuracy 更适合梯度下降. 但它并不自动提升 generalization; 若模型在训练集上把概率推得过于自信, 仍可能过拟合.

3. 标签平滑与 margin 类损失

- 标签平滑 (Label Smoothing) 把 one-hot 标签替换为较平滑的分布:

$$y_c^{LS} = (1 - \epsilon)y_c + \frac{\epsilon}{C}.$$

这会降低模型对单一类别输出 100% 置信度的冲动, 在分类任务中常用作正则化手段.

- Hinge Loss 等 margin 类损失不直接拟合概率, 而是要求正确类别得分比错误类别高出一定间隔. 它体现了另一类设计思想: loss 不只为可优化服务, 也可把泛化偏好写进训练目标.

4. 更多数据与数据分布代表性

- 若 overfitting 来自训练集不能代表真实分布, 最直接的办法是收集更多有代表性的数据. 这相当于改变经验风险中的求和样本, 因而属于 loss 定义的一部分.
- 更多数据主要改善 generalization, 通常不会让 optimization 更容易. 数据越多, 模型需要同时满足的约束越多, 训练过程反而可能更难.
- 因此, 当 training loss 本来就降不下去时, 盲目增加数据通常不是第一优先级; 当 training loss 很低但 validation loss 很差时, 增加有代表性的数据才更直接.

5. 数据增强与 Mixup

- 数据增强 (Data Augmentation) 通过任务保持的变换构造额外训练样本. 图像分类中常见翻转、裁剪、颜色扰动、模糊等; 语音任务中可使用音量变化、速度变化或噪声扰动.
- 增强必须符合任务语义. 若任务是判断鸟头朝向, 左右翻转会改变标签, 不能保留原标签; 若任务是说话人识别, 把声音转换成另一个人的音色后仍保留原说话人标签也是错误增强.

- Mixup 同时混合输入和标签. 对两笔样本 $(\mathbf{x}_i, \mathbf{y}_i)$ 与 $(\mathbf{x}_j, \mathbf{y}_j)$, 取 $\lambda \in [0, 1]$, 构造

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j, \quad \tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j.$$

它迫使模型在样本之间的线性插值区域给出平滑预测, 通常用于改善 generalization.

6. 半监督学习中的无标签数据

- 无标签数据可以通过额外 loss 项参与训练:

$$\mathcal{L} = \mathcal{L}_{labeled} + \lambda \mathcal{L}_{unlabeled}.$$

关键问题是如何定义不依赖真实标签的 $\mathcal{L}_{unlabeled}$.

- 一种思路是熵最小化. 对无标签样本 $\mathbf{x}$, 模型输出分布 $\mathbf{p} = f(\mathbf{x})$, 可鼓励

$$H(\mathbf{p}) = - \sum_c p_c \log p_c$$

尽可能小. 这表达了“样本应更明确地属于某一类”的假设.

- 另一种思路是相似样本预测一致. 若 $G(\mathbf{x}, \mathbf{x}')$ 表示两样本在图上相连, 可加入

$$\sum_{\mathbf{x}, \mathbf{x}'} G(\mathbf{x}, \mathbf{x}') D(f(\mathbf{x}), f(\mathbf{x}')),$$

其中 D 可取 KL 散度、交叉熵或其他分布距离. 这表达了“相似输入应有相似输出”的假设.

- 半监督学习主要服务于 generalization. 它让 loss 更复杂, 并不会自动降低 optimization 难度.

问题思考:

- 为什么 accuracy 适合报告分类效果, 却不适合直接作为梯度下降的训练 loss?
- 数据增强是否总是安全? 请举出一个增强会改变标签语义的例子.
- 熵最小化和相似样本一致性分别依赖什么关于数据分布的假设?

8 超参数优化

本节导读:

- **本节目的:** 把“调参”从凭感觉试错变成可复现的模型选择流程.
- **为什么学:** 学习率、batch size、模型容量、正则化强度和训练预算之间相互耦合, 只看一次实验结果很容易误判.
- **核心内容:** 超参数优化是在固定数据划分和评价协议下, 用训练集学习参数, 用验证集选择配置, 最后才用测试集报告最终性能.

- **最小例子**: 网格搜索可以直接拿来用: 先列出候选学习率 $\{10^{-4}, 10^{-3}, 10^{-2}\}$ 和候选 weight decay $\{0, 10^{-4}, 10^{-3}\}$, 然后训练所有 $3 \times 3 = 9$ 组配置, 在同一个验证集上选择表现最好的一组. 这个方法简单、可复现, 但候选维度一多, 实验数量会迅速增加.
- **最该记住**: 测试集不能参与搜索, 跨数量级变化的超参数应优先在对数尺度上搜索, 每次实验都要记录配置、预算、随机性和指标.

优化器、学习率、batch size、正则化强度和网络结构都会显著影响最终结果. 超参数优化的目标不是在测试集上反复试错, 而是在训练集和验证集之间建立规范的模型选择流程.

1. 常见超参数

- 优化相关超参数包括学习率、学习率调度、optimizer 类型、momentum 系数、Adam 的 β_1, β_2 、batch size 和梯度截断阈值.
- 正则化相关超参数包括 weight decay 系数、Dropout 比例、label smoothing 系数、数据增强强度和 early stopping patience.
- 结构相关超参数包括层数、隐藏单元数、卷积通道数、kernel size、stride、归一化方式、激活函数和残差块数量.

2. 搜索策略

- 网格搜索 (Grid Search) 适合维度较低、每个超参数候选较少的场景, 优点是简单可复现. 假设共有 K 个超参数, 第 k 个超参数有 m_k 个候选值, 则总实验数为

$$\prod_{k=1}^K m_k.$$

若某个超参数是连续变量, 通常需要先离散化为若干候选点. 例如学习率可在对数尺度上取

$$\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}\}.$$

网格搜索在小规模实验中仍然有价值, 因为它给出了清晰的对照表; 但当维度增加时, 实验数量会迅速变大.

- 随机搜索 (Random Search) 常比网格搜索更有效, 因为高维超参数空间通常只有少数维度真正敏感. 用一个二维函数作直观说明: 若目标近似为 $f(x, y) = g(x) + h(y)$ 且主要由 $g(x)$ 决定, 3×3 网格虽然做了 9 次实验, 但只测试了 x 的 3 个取值; 随机搜索做 9 次实验时, 通常会覆盖 x 的更多不同取值.
- 因此, 对学习率、weight decay、Dropout 比例等敏感超参数, 随机搜索往往能更快发现可用区域. 对学习率、weight decay 这类跨数量级变化的正数, 应优先在对数空间采样, 而不是在线性空间均匀采样.
- 贝叶斯优化用代理模型预测哪些配置更值得尝试, 适合单次训练成本较高的场景. Hyperband 和 Successive Halving 则通过提前停止表现差的配置来节省资源.
- 神经架构搜索 (NAS) 把结构设计也纳入搜索, 但成本较高, 对课程实验而言更重要的是理解搜索原则, 而不是盲目追求自动化.

3. 验证协议与实验记录

- 每次比较超参数时, 应固定数据划分、随机种子或至少记录随机性来源, 否则很难判断差异来自方法本身还是实验噪声.
- 最终报告应区分 validation performance 和 test performance. 验证集用于选择配置, 测试集用于最终评估.
- 训练曲线应同时记录 training loss、validation loss 和任务评价指标. 只看最终 accuracy 容易掩盖 optimization 失败、过拟合和学习率不合适等问题.
- 学习曲线还可以用于提前判断一组配置是否值得继续训练. 若 training loss 不收敛、很快发散, 或 validation performance 长时间明显落后于已有配置, 可以用 early stopping 或 Successive Halving 中止该实验, 把计算预算留给更有希望的配置.
- 需要注意, 用学习曲线提前停止是资源分配策略, 不等同于最终模型训练中的 early stopping 正则化. 做超参数比较时, 应记录每个配置的训练预算、停止条件和最终指标, 否则不同配置之间的比较会不公平.

4. 一个实用调参顺序

- 先让模型在小规模数据上能够过拟合, 以确认实现、loss 和反向传播没有明显错误.
- 再调学习率和 optimizer, 让 training loss 能够稳定下降.
- 然后调模型容量和归一化方式, 观察是否存在欠拟合或优化困难.
- 最后加入正则化、数据增强和 early stopping, 缩小 training 与 validation 的差距.

问题思考:

- 为什么学习率通常要在对数尺度上搜索?
- 如果一个配置 validation loss 更低但 training loss 更高, 这可能说明什么?
- 为什么测试集不能参与超参数搜索?

9 AutoResearch: 用 Agent 自动化实验与调参

本节导读:

- **本节目的:** 把前面讲过的优化、正则化和超参数搜索放进一种自动化研究 workflows 中, 理解 Agent 能自动化哪些部分, 不能替代哪些判断.
- **为什么学:** 现代深度学习实验的搜索空间很大, 许多重复性的改代码、跑训练、读指标、记录结果和回滚无效改动可以被流程化.
- **核心内容:** AutoResearch 自动化的不是新的学习理论, 而是一个受约束的实验闭环, 即在固定预算和验证指标下提出改动、执行训练、比较结果并保留或丢弃.
- **最小例子:** 让 Agent 只允许修改 `train.py`, 每次固定训练一小段时间, 再用一批没有参与训练的数据检查预测损失是否下降. 如果结果变好, 就把这次代码改动保存成一个版本记录, 也就是一次 commit; 如果结果变差或程序崩溃, 就回到上一版并记录原因.
- **最该记住:** Agent 可以提高实验吞吐量, 但实验边界、验证协议、测试集复核和复杂度控制仍然必须由研究者负责.

前面几节讨论了优化算法、正则化、验证集和超参数搜索。这些内容仍然假设研究者手动提出实验、修改代码、运行训练并比较结果。AutoResearch 代表一种更现代的研究工作流：让 Agent 在人类设定的边界内自动提出和执行实验，用验证集指标判断是否保留改动，从而把大量重复性的调参和消融实验自动化。

1. 为什么需要 AutoResearch

- 深度学习实验空间很大。学习率、batch size、optimizer、weight decay、模型层数、隐藏维度、归一化方式和训练调度都可能相互影响。人工逐个试验很容易遗漏组合，也很难长时间保持一致的实验记录。
- 传统超参数优化通常把实验看成搜索问题，例如网格搜索、随机搜索或贝叶斯优化。AutoResearch 更进一步：它不只搜索几个数值超参数，还允许 Agent 修改训练代码中的结构、优化器和训练循环，因而更接近真实研究过程。
- 这种方式适合的并不是“让机器替代判断”，而是把已经规范化的实验循环交给 Agent 执行。人类仍然要定义任务、数据、指标、约束和可修改范围，并最终判断哪些结果值得相信。

2. 与前面内容的关系

- 从 optimization 角度看，Agent 可以尝试不同学习率、batch size、warmup、学习率衰减、Momentum/AdamW 等配置，观察训练集损失和验证集损失的变化。
- 从 generalization 角度看，Agent 可以尝试 weight decay、Dropout、数据增强或模型容量调整，但仍必须通过验证集判断是否真的改善泛化，不能只看训练集损失。
- 从实验方法角度看，AutoResearch 依赖前面已经强调的验证协议：固定训练预算，固定评价指标，记录每次实验，保留带来验证集改进的改动，回滚无效或失败的改动。
- 因此，AutoResearch 不是新的梯度下降算法。它是在优化算法和超参数优化之上的自动化研究框架，目标是提高实验迭代速度和实验管理质量。

3. karpathy/autoresearch 的核心设计

- 公开项目 <https://github.com/karpathy/autoresearch> 的入口思想可以概括为：给 Agent 一个小而真实的语言模型训练系统，让它在夜间持续实验。每次实验修改训练代码，训练固定 5 分钟，用验证集指标判断是否变好，好则保留，不好则丢弃。
- 项目刻意保持很小，主要由三个文件组成：
 - `prepare.py`: 固定数据准备、tokenizer、dataloader 和评价函数。其中 `TIME_BUDGET=300` 秒表示每次实验只训练 5 分钟；评价指标是验证集上的预测损失，代码中记为 `val_bpb`，可简单理解为“模型在没见过的数据上预测得有多好”，数值越低越好。
 - `train.py`: Agent 唯一允许修改的核心文件。它包含模型、optimizer、batch size、学习率、训练循环和最终打印的实验摘要。
 - `program.md`: 人类写给 Agent 的实验规则。它规定先跑 baseline，再循环执行“修改 `train.py`、提交、训练、读取指标、记录结果、保留或回滚”。
- 固定 5 分钟训练预算是一个关键设计。如果某次改动让模型更大，它可能单步更强但训练步数更少；如果某次改动让模型更小，它可能单步更弱但跑得更快。固定真实耗时后，比较的是“在同一算力和时间约束下谁能在没见过的数据上预测得更好”。

- `program.md` 还要求把每次实验写入 `results.tsv`, 包括这次代码版本记录 (commit)、验证集预测损失、显存、状态和实验描述. 若指标改善, 就保留这次代码版本; 若指标变差或程序崩溃, 就回到上一版. 这相当于把研究过程做成一个可复现的闭环:

提出想法 → 改代码 → 固定预算训练 → 验证集评价 → 记录并保留/回滚.

4. 基本原则与局限

- 该项目中的 GPT 结构、Flash Attention 或 Muon optimizer 并不是理解 AutoResearch 的必要前提. 更核心的是实验闭环: Agent 自动执行的仍然是训练、验证、调参和模型选择, 只是把这些步骤组织成连续运行的流程.
- AutoResearch 的关键不在“Agent 很聪明”, 而在实验边界设计得足够清楚: 哪些文件能改, 哪些函数不能改, 用什么指标比较, 一次训练给多少预算, 什么情况算成功, 什么情况必须回滚.
- 在近期相关研究工作中, 也可以使用类似的 Agent 化实验框架: 研究者负责提出研究目标和约束, Agent 负责自动生成候选实验、运行训练、整理日志和筛掉无效配置. 这种方式不会消除研究判断, 但可以显著提高实验吞吐量.
- 需要注意, AutoResearch 也可能过拟合验证集、引入复杂但收益很小的改动, 或因为评价指标设计不当而优化错方向. 因此, 最终仍需要测试集评估、人工复核和对实验复杂度的控制.

问题思考:

- AutoResearch 自动化的是优化算法本身, 还是研究者的实验循环? 二者有什么区别?
- 为什么固定训练时间和固定验证指标对自动化实验很重要?
- 如果 Agent 连续根据验证集结果保留改动, 可能会带来什么新的过拟合风险?

(注: 详细定义和更多内容请参考课程教材第 7 章)